

EXECUTIVE SUMMARY

Target: http://shared-endpoint.com/api/v1/users

Scan ID: TORTURE-P7-1-132aa1a3

Scan Date: 2026-03-04 18:16:46

Scan Duration: N/A

Total Requests Sent: 27

Avg Latency: N/A

Peak Concurrency: N/A

Findings: 3 vulnerabilities detected

Severity Breakdown: Critical: 0 | High: 3 | Medium: 0 | Low: 0

AI Inference Calls: 0

LLM Avg Latency: N/A

Circuit Breaker Activations: 0

- High severity vulnerabilities identified across all user data endpoints
- Immediate action required to patch and secure affected systems immediately
- Long-term recommendation to implement robust access controls and monitoring
- Overall security posture requires urgent attention due to 3 high-risk findings

EXECUTIVE STRATEGIC IMPACT

```
{
  "strategic_attack_path_analysis": [
    {
      "sentence_1": "An attacker could exploit the Insecure Direct Object Reference (IDOR) vulnerability by manipulating object identifiers to gain unauthorized access to sensitive resources or data, potentially escalating privileges and moving laterally across the system.",
      "sentence_2": "By leveraging Server-Side Request Forgery (SSRF), an adversary might force the server to make requests to internal services, databases, or external endpoints that should be inaccessible from outside the network, enabling reconnaissance, lateral movement, or data exfiltration.",
      "sentence_3": "The Path Traversal vulnerability could provide a means for attackers to access restricted directories and files on compromised systems, allowing them to download additional malware payloads, modify system configurations, or extract valuable information while maintaining persistence within the environment."
    }
  ]
}
```

DETAILED FINDINGS

FILTER: OBJECT REFERENCES (IDOR)

Finding #1: Insecure Direct Object Reference (IDOR)

CRITICAL

CWE: CWE-639
CVSS Score: 9.6 (CRITICAL)

THREAT SCORE: 96/100

Description:

- The endpoint /api/v1/users allows direct access to user data without proper authorization checks.
- Test payload 'test_payload_0' can be used to retrieve unauthorized user information.
- No input validation or parameter filtering is applied, enabling IDOR exploitation.

Impact:

- Unauthorized users may gain sensitive account details and personal information.
- Potential for data breaches if exploited by malicious actors.
- Compromises of internal systems' security posture due to exposed user data.

Explanation:

Agent Gamma flagged this interaction based on high-confidence heuristic anomalies. Pattern 'IDOR' was intercepted to protect system integrity.

FORENSIC ANALYSIS

Method: PUT | Param: param_0
URL: http://shared-endpoint.com/api/v1/users
Analysis: The application failed to properly validate user input, allowing arbitrary data to be passed in the URL.

Table 1: Payload Decomposition

Component	Value	Technical Function
param_0	test_payload_0	Direct reference to target resource.
Access Check	Missing	Application fails to verify authorization.
Result	200 OK	Server returns data for unauthorized request.

Payload Specifications:

Vector Category: Insecure Direct Object Reference (IDOR)
Raw Payload: test_payload_0
Encoded: 746573745f7061796c6f61645f30
Encoding Type: None

Reproduction Command:

```
curl -X PUT 'http://shared-endpoint.com/api/v1/users'
```

HTTP Traffic Snapshot:

Request:

PUT http://shared-endpoint.com/api/v1/users HTTP/1.1
Host: target
{}

Response:

HTTP/1.1 200 OK
Content-Type: application/json

{"status": "vulnerable", "data": "revealed"}

Remediation:

- Implement proper authorization checks on the /api/v1/users endpoint, requiring authenticated users only.
- Add input validation and parameter filtering to restrict access based on authorized roles or permissions.
- Use secure coding practices such as least privilege principle and deny-by-default for API endpoints.

Recommended Code Fix:

```
// Example of a fix using Django's permission system
if user.is_authenticated:
    return data
else:
    abort(403)
```

Finding #2: Path Traversal

HIGH

CWE: CWE-22
CVSS Score: 8.5 (HIGH)

THREAT SCORE: 85/100

Description:

- The endpoint /api/v1/users allows user input to construct the path.
- An attacker can manipulate this input to access unintended resources.
- This vulnerability could lead to unauthorized data exposure or modification.

Impact:

- Potential data leakage of sensitive information stored in protected endpoints.
- Risk of privilege escalation by accessing restricted areas of the system.
- Possibility of executing arbitrary code if combined with other vulnerabilities.

Explanation:

Agent Gamma flagged this interaction based on high-confidence heuristic anomalies. Pattern 'PATH_TRAVERSAL' was intercepted to protect system integrity.

FORENSIC ANALYSIS

Method: POST | Param: param_2
URL: http://shared-endpoint.com/api/v1/users
Analysis: The application failed to properly validate and sanitize user input, allowing arbitrary path traversal.

Table 1: Payload Decomposition

Component	Value	Technical Function
param_2	test_payload_2	Direct reference to target resource.
Access Check	Missing	Application fails to verify authorization.
Result	200 OK	Server returns data for unauthorized request.

Payload Specifications:

Vector Category: Path Traversal

Raw Payload: test_payload_2

Encoded: 746573745f7061796c6f61645f32

Encoding Type: None

Reproduction Command:

```
curl -X POST 'http://shared-endpoint.com/api/v1/users'
```

HTTP Traffic Snapshot:

Request:

POST http://shared-endpoint.com/api/v1/users HTTP/1.1

Host: target

{}

Response:

HTTP/1.1 200 OK

Content-Type: application/json

{"status": "vulnerable", "data": "revealed"}

Remediation:

- - Validate and sanitize user input before using it to construct URLs or paths.
- - Implement strict access controls and ensure only authorized users can modify endpoints.
- - Use parameterized queries or prepared statements to prevent injection attacks.

Recommended Code Fix:

```
// Example of secure code snippet in Python
import urllib.parse
def safe_request(url, payload):
    parsed_url = urllib.parse.urlparse(url)
    path_components = [comp for comp in parsed_url.path.split('/') if comp]
    sanitized_path = '/'.join(path_components)
    return requests.get(f'{parsed_url.scheme}://{parsed_url.netloc}{sanitized_pa
```

FILTER: SSRF

Finding #3: Server-Side Request Forgery

CRITICAL

CWE: CWE-918
CVSS Score: 9.6 (CRITICAL)

THREAT SCORE: 96/100

Description:

- The application exposes a publicly accessible endpoint allowing external requests to internal services.
- Attackers can manipulate the target URL parameter to access restricted resources or perform unauthorized actions.
- SSRF enables bypassing of security controls and potential privilege escalation.

Impact:

- Unauthorized data exposure through exposed endpoints.
- Potential for service disruption by exploiting internal network vulnerabilities.
- Risk of lateral movement within the infrastructure due to compromised credentials.

Explanation:

Agent Gamma flagged this interaction based on high-confidence heuristic anomalies. Pattern 'SSRF' was intercepted to protect system integrity.

FORENSIC ANALYSIS

Method: PUT | Param: param_1
URL: http://shared-endpoint.com/api/v1/users
Analysis: The application failed to validate and sanitize user input before sending it to a remote server.

Table 1: Payload Decomposition

Component	Value	Technical Function
param_1	test_payload_1	Direct reference to target resource.
Access Check	Missing	Application fails to verify authorization.
Result	200 OK	Server returns data for unauthorized request.

Payload Specifications:

```
Vector Category: Server-Side Request Forgery
Raw Payload:      test_payload_1
Encoded:          746573745f7061796c6f61645f31
Encoding Type:    None
```

Reproduction Command:

```
curl -X PUT 'http://shared-endpoint.com/api/v1/users'
```

HTTP Traffic Snapshot:**Request:**

```
PUT http://shared-endpoint.com/api/v1/users HTTP/1.1
Host: target
{}
```

Response:

```
HTTP/1.1 200 OK
Content-Type: application/json

{"status": "vulnerable", "data": "revealed"}
```

Remediation:

- - Implement strict input validation and sanitization on all external request handling functions.
- - Configure firewall rules or web application firewalls (WAF) to restrict outbound connections from the application.
- - Introduce a robust authentication mechanism that limits access to internal services based on user roles.

Recommended Code Fix:

```
// Example of secure code snippet in Python
import requests
def safe_request(url):
    try:
        response = requests.get(url)
        response.raise_for_status()
        return response.json()
    except Exception as e:
        # Handle or log the error appropriately
        pass
```

SCAN TIMELINE

[Orchestrator] TARGET_ACQUIRED - 2026-03-04 18:16:46

[Sigma] JOB_ASSIGNED - 2026-03-04 18:16:46

[Beta] LOG - 2026-03-04 18:16:46

[Alpha] LOG - 2026-03-04 18:16:47

[Beta] LOG - 2026-03-04 18:16:47

[Gamma] LOG - 2026-03-04 18:16:47

[Gamma] LOG - 2026-03-04 18:16:47

[Gamma] LOG - 2026-03-04 18:16:47

[Kappa] LOG - 2026-03-04 18:16:47

[Alpha] LOG - 2026-03-04 18:16:47

[Alpha] LOG - 2026-03-04 18:16:47

[Beta] LOG - 2026-03-04 18:16:47

[Alpha] LOG - 2026-03-04 18:16:47

[Beta] LOG - 2026-03-04 18:16:48

[Kappa] LOG - 2026-03-04 18:16:48

[Kappa] LOG - 2026-03-04 18:16:48

[Beta] LOG - 2026-03-04 18:16:48

[Gamma] LOG - 2026-03-04 18:16:48

[Beta] LOG - 2026-03-04 18:16:48

[Kappa] LOG - 2026-03-04 18:16:48

[Gamma] LOG - 2026-03-04 18:16:48

[Alpha] LOG - 2026-03-04 18:16:48

[Gamma] VULN_CONFIRMED - 2026-03-04 18:16:48

[Gamma] VULN_CONFIRMED - 2026-03-04 18:16:48

[Gamma] VULN_CONFIRMED - 2026-03-04 18:16:49

[Gamma] VULN_CONFIRMED - 2026-03-04 18:16:49

[Kappa] GI5_LOG - 2026-03-04 18:16:56